ASANNI

Customer Mobile Application & Restaurant Desktop Software

Technical Collaboration Document

Prepared by: Anas Afzal — Android Application Developer
Project: ASANNI — Restaurant Discovery and Table Availability Platform
Mobile Platform: Android (Java, Android Studio)
Desktop Platform: To be determined by the Restaurant Phase developer(Shariq Gazdar)
Project Stage: Version 1.1.0 — Current development and integration planning

---

1. Introduction: What Is ASANNI?

ASANNI is a platform designed to improve the dining experience in Karachi by connecting customers with restaurants through real-time information about table availability, estimated waiting times, and reservations.

The primary problem ASANNI aims to solve is simple: customers often travel to a restaurant without knowing whether a table is available. They may have to wait in queues, search multiple restaurants, or change their plans at the last minute.

ASANNI aims to make this process more convenient by allowing customers to discover restaurants, check availability, and plan their visits before leaving home.

The system consists of two major applications:

1. **Customer Phase — Android Mobile Application:** Developed by Anas Afzal. This application serves customers who want to discover restaurants, check table availability, and make reservations.
2. **Restaurant Phase — Desktop Software:** Developed by the Shariq Gazdar. This application serves restaurant owners and staff who need to manage their restaurants, tables, availability, reservations, and other operational information.

Although these applications have different interfaces and users, they are two parts of the same system. They must eventually exchange information through a shared backend, database, or API.

The mobile application is not intended to operate as an isolated project. Its purpose is to provide customers with information that originates from the restaurant management side.

2. My Responsibilities: Customer Mobile Application

My responsibility is to develop the Android application that customers will use to interact with ASANNI.

I am using **Java in Android Studio**. My current focus is the mobile user interface, navigation, customer-facing functionality, and the structure needed to connect the application with the rest of the platform.

The mobile application will guide a customer through the following general journey:

Open ASANNI → Get Started → Discover Restaurants → View Restaurant Details → Check Availability → Reserve a Table → Receive Reservation Information

The precise features available in the first release will depend on the agreed project scope and the backend integration.

2.1. Application Launch and Onboarding

The first experience a customer has with ASANNI should be visually appealing, simple, and easy to understand.

The onboarding experience may include:

- A launch screen displaying the ASANNI identity.
- A Get Started page introducing the purpose of the application.
- Appropriate illustrations or restaurant-related imagery.
- Short descriptions of the platform's benefits.
- Smooth transitions and subtle animations.
- Clear navigation to the next step.

The design should be consistent throughout the application, including typography, colours, buttons, icons, spacing, and animations.

I intend to keep the interface organized so that images, text, colours, and other visual elements can be modified without unnecessarily rewriting the application's functionality.

2.2. Customer Registration and Authentication

Customers may need an account to make and manage reservations.

The authentication flow should support:

- Creating a customer account.

- Logging in and logging out.
- Validating required information.
- Displaying meaningful errors when authentication fails.
- Maintaining an authenticated session where appropriate.

The exact authentication method, required fields, and password recovery process need to be agreed upon.

If registration and authentication are included in the first release, the mobile app must communicate with the shared backend rather than maintaining an independent list of customers.

A customer's identity should be associated with their reservations so that they can retrieve their own booking information.

2.3. Customer Home Screen

The home screen will be the main entry point into the restaurant discovery experience.

Its purpose is to help customers find a suitable restaurant without navigating through unnecessary menus.

Potential elements include:

- A search bar for finding restaurants.
- A restaurant list with images and essential details.
- Location-based restaurant discovery.
- Availability information.
- Estimated waiting times where the restaurant supplies the necessary information.
- Filters for narrowing down restaurant choices.
- Access to reservations and customer account information.

The layout should prioritize useful information rather than overcrowding the screen with decorative elements.

The home screen should also handle empty states gracefully. For example, if there are no restaurants available in the selected area, the application should explain the situation rather than displaying a blank page.

2.4. Location-Based Restaurant Discovery

ASANNI is intended to help customers discover restaurants around them.

The Android application may request location permission to identify nearby restaurants and improve the relevance of search results.

The location functionality should include:

- Requesting permission when the feature requires it.
- Explaining why location access is useful.
- Handling denied or unavailable permissions.
- Allowing restaurant discovery without location access through an alternative, such as manual search.
- Displaying relevant restaurant information based on the selected location.

The mobile application can determine the customer's location, but it cannot independently know whether a restaurant currently has available tables.

Restaurant availability must come from the restaurant management system through the shared backend.

This distinction is important: **location identifies where the customer is; restaurant data identifies which restaurants exist; availability data identifies whether they can accommodate the customer.**

2.5. Restaurant Listing and Details

When a customer selects a restaurant, the application should display a dedicated details screen.

Depending on the information provided by the restaurant management system, the screen may include:

- Restaurant name and photographs.
- Address and location.
- Cuisine or restaurant category.
- Opening hours.
- Contact information.
- Available facilities or other restaurant information.
- Current table availability.
- Estimated waiting time, where supported.
- An option to proceed with a reservation.

The application must distinguish between information that is relatively static, such as a restaurant's name, and information that changes frequently, such as table availability.

Static information can be updated less frequently, whereas availability should be refreshed from the backend whenever appropriate.

The interface must not display invented availability or imply that a reservation is confirmed simply because the customer has opened the restaurant details page.

2.6. Live Table Availability and Waiting Times

This is one of the most important features of ASANNI.

The customer should be able to determine whether a restaurant can accommodate them before travelling there.

For this feature to work, the restaurant desktop software must maintain or update the restaurant's table and availability information. The backend must then make the relevant information available to the Android application.

The mobile application should be able to display information such as:

- Whether the restaurant is currently accepting customers.
- Whether suitable tables are available.
- The availability of tables for a requested party size, if supported.
- An estimated waiting time when a suitable table is not immediately available.
- The time at which the information was last updated, where useful.

For example, imagine a customer wants a table for four people.

1. The customer opens a restaurant in the mobile app.
2. The app requests current availability from the backend.
3. The backend checks the information maintained by the restaurant management system.
4. The app displays the returned availability information.
5. The customer decides whether to visit immediately or make a reservation.

A crucial technical consideration is that **the mobile application must not be the source of truth for restaurant availability**. The backend and restaurant management system must coordinate the relevant data.

If availability changes after the customer opens the page, the app should refresh the information or clearly indicate that the information may be outdated.

For the first version, availability could be retrieved through regular API requests. More advanced live updates could be considered later if the agreed infrastructure supports them.

3. Table Reservation System

The reservation system is the point at which the customer-facing and restaurant-facing applications become especially dependent on one another.

3.1. Customer Reservation Flow

The proposed customer flow is:

1. The customer selects a restaurant.
2. The customer chooses a suitable date and time, if reservations for future times are supported.
3. The customer enters the number of people.
4. The application requests availability for the selected requirements.
5. The customer submits a reservation request.
6. The backend validates the request against the restaurant's current availability.
7. The backend creates the reservation and returns its status.
8. The mobile application displays the result.

The reservation status must be determined by the backend, not by the mobile application's button animation or local screen state.

For example, if two customers attempt to reserve the same remaining table at nearly the same time, the system must prevent both requests from successfully claiming the same capacity.

This requires server-side validation and a reliable way of updating reservation and table availability information.

3.2. Reservation Confirmation

After a successful reservation, the application should display the relevant booking details, potentially including:

- Reservation reference number.
- Restaurant name.
- Date and time.
- Number of guests.
- Reservation status.
- Any additional instructions supplied by the restaurant.

The interface should distinguish between a submitted request, a pending reservation, and a confirmed reservation if the backend supports those states.

A reservation should only be presented as confirmed when the backend has returned a successful confirmation.

3.3. My Reservations

Customers should have a dedicated area where they can view their reservations.

Depending on the agreed functionality, this section may include:

- Upcoming reservations.
- Past reservations.
- Reservation details.
- Current reservation status.
- Cancellation options.
- Updated information if the restaurant changes the reservation.

If cancellation is supported, the mobile app should submit the cancellation request to the backend. It should not simply remove the reservation from the customer's screen while leaving the restaurant's records unchanged.

The restaurant desktop software should receive the corresponding status update.

4. Restaurant Desktop Software: Requirements:

Your application serves a different purpose from mine.

While my application helps customers discover and reserve tables, your application provides the tools through which restaurant staff manage the information that customers see.

The following describes the expected relationship between the applications. These are proposed functional requirements, not assumptions about features you have already implemented.

4.1. Restaurant Management

The desktop application may allow authorized restaurant staff to manage:

- Restaurant profile information.
- Restaurant name, address, and contact information.
- Opening hours.
- Restaurant categories and other details.
- Relevant information displayed to customers.

When restaurant information is updated, the changes should eventually be reflected in the customer application through the backend.

4.2. Table Management

The restaurant software should maintain information about the restaurant's seating capacity and table availability.

Depending on the restaurant's workflow, this could include:

- Individual tables and their capacities.
- Tables currently occupied.
- Tables available for new customers.
- Reservations associated with particular tables or seating capacity.
- Tables temporarily unavailable.
- Changes in table status made by restaurant staff.

The implementation should account for the restaurant's actual seating workflow rather than assuming that every restaurant manages tables in exactly the same way.

For example, a table for six might accommodate a party of four, whereas two separate tables for two may or may not be combinable. The system's rules need to be agreed upon.

4.3. Reservation Management

The desktop software should provide restaurant staff with a way to review and manage customer reservations.

Potential functionality includes:

- Viewing incoming reservations.
- Viewing reservation details.
- Accepting or rejecting requests if manual approval is part of the workflow.
- Confirming arrivals.
- Recording cancellations.
- Updating reservation status.
- Keeping availability consistent with reservation changes.

The customer should not need to contact the restaurant separately merely to find out whether an online reservation has been recorded, provided the system supports the relevant confirmation workflow.

4.4. Availability and Waiting-Time Updates

Restaurant staff should be able to update the information required to calculate or display current availability.

Waiting-time estimates need particular attention. They can depend on occupancy, party size, expected table turnover, and the restaurant's own procedures.

We should agree whether waiting times will initially be entered manually by staff, calculated by the system, or provided through a combination of both methods.

Regardless of the approach, the mobile app should display the information supplied by the agreed backend logic and should not independently invent waiting-time estimates.

5. How Our Applications Will Communicate

The mobile app and desktop software should not operate as two completely separate systems.

They need a shared way to exchange information. The proposed architecture is:

Android Customer App ↔ Shared Backend/API ↔ Database ↔ Restaurant Desktop Software

The precise implementation may differ depending on your desktop technology and the team's backend decisions.

5.1. Why We Need a Shared Backend

Consider what happens when restaurant staff mark a table as occupied.

If the desktop software only changes its own local database, the customer app cannot reliably know that the table is occupied.

Likewise, if a customer makes a reservation only inside the Android app, restaurant staff will not necessarily see it.

A shared backend solves this coordination problem by providing a common source of information.

The general flow is:

1. Restaurant staff update information in the desktop software.
2. The desktop software sends the change to the backend.
3. The backend validates and stores the information.
4. The customer app requests the updated information.
5. The customer sees the current information returned by the backend.

The reverse flow also matters:

1. A customer submits a reservation from Android.
2. The backend validates the request and records the reservation.
3. The desktop software retrieves or receives the new reservation.
4. Restaurant staff see the booking and manage it.

5. Any subsequent changes are reflected in the customer app.

The backend should enforce authorization, data validation, and consistency for both applications.

5.2. API Communication

One possible approach is to expose a REST API over HTTPS.

The API would provide endpoints for the operations that both applications need, such as:

- Retrieving restaurant listings.
- Retrieving restaurant details.
- Searching by location or other criteria.
- Checking table availability.
- Creating reservations.
- Retrieving a customer's reservations.
- Updating reservation status.
- Updating restaurant and table information.

These are examples of required API capabilities, not existing endpoint names. The actual URLs, request formats, response formats, and authentication method need to be agreed upon before integration.

For example, an availability request might conceptually contain:

- Restaurant ID.
- Requested date and time.
- Number of guests.

The response could contain:

- Whether the requested seating is available.
- Relevant availability information.
- An estimated waiting time, if supported.
- A timestamp indicating when the information was updated.

The API must validate the request and return meaningful errors when the information is invalid or unavailable.

5.3. Proposed Shared Data Models

Both applications should agree on the core data structures.

Restaurant

- Restaurant ID.
- Name.
- Description.
- Address and coordinates, where available.
- Contact details.
- Opening hours.
- Other customer-visible details.

Table

- Table ID.
- Restaurant ID.
- Seating capacity.
- Current status.
- Any additional information required by the restaurant's table-management rules.

Customer

- Customer ID.
- Name and other agreed profile fields.
- Authentication identity managed securely by the backend.

Reservation

- Reservation ID.
- Customer ID.
- Restaurant ID.
- Date and time.
- Number of guests.
- Reservation status.
- Relevant timestamps and other agreed booking details.

Availability

- Restaurant ID or relevant table references.
- Requested or observed time.
- Available capacity or seating options.

- Waiting-time estimate, if applicable.
- Last-updated timestamp.

These are logical models. The actual database schema may contain additional fields, and some fields may be calculated rather than stored.

The most important requirement is that both applications interpret identifiers, statuses, dates, and times consistently.

6. My Android Application Architecture

I am developing the Android application in Java using Android Studio.

To keep the project maintainable, I intend to organize the code by responsibility and feature rather than placing all functionality inside one large Activity.

The proposed structure includes the following areas:

Core

- Shared utilities.
- Common UI components.
- Network configuration.
- Location-related functionality.

Data

- Data models.
- API communication.
- Repositories that provide data to the application's features.

Features

- Authentication.
- Restaurant discovery and details.
- Availability.
- Reservations.
- Customer profile and account functionality.

The precise package structure may change as implementation progresses, but the goal is to separate the interface from the data and business logic.

For example, the restaurant details screen should not contain all the networking code, reservation rules, and data-processing logic itself. It should request the information it needs through the appropriate application components.

This separation will make the app easier to debug, maintain, and extend in future releases.

7. Interface Design and User Experience

The application should feel like one coherent product rather than a collection of unrelated screens.

My UI development priorities are:

- A consistent design system.
- Clear and intuitive navigation.
- Smooth screen transitions and subtle animations.
- Responsive layouts for different Android screen sizes.
- Readable typography and appropriate spacing.
- Loading indicators during network requests.
- Helpful error messages.
- Empty states when no results are available.
- Clear feedback after customer actions.
- Accessible touch targets and readable contrast.

Animations should enhance the experience without making the application slow or distracting.

The underlying functionality must also remain reliable. A beautiful reservation screen is not useful if it displays stale availability or confirms a booking that the backend has rejected.

I will keep the presentation layer separate from the business logic so that design improvements do not unnecessarily disrupt the core functionality.

8. Important Integration Rules

To avoid inconsistencies between the mobile app and desktop software, we should establish a few rules before connecting the applications.

Rule 1 — One authoritative reservation record

A reservation should be recorded and validated by the shared backend, rather than existing independently in each application's local storage.

Rule 2 — Consistent reservation statuses

Both applications must agree on what each reservation status means. For example, the meaning and allowed transitions of pending, confirmed, cancelled, and completed reservations must be defined.

Rule 3 — Server-side availability validation

The backend must recheck availability when a customer submits a reservation. A previously displayed table may no longer be available.

Rule 4 — Consistent identifiers

Restaurant IDs, table IDs, customer IDs, and reservation IDs must remain consistent across both applications.

Rule 5 — Consistent date and time handling

Both applications must agree on how dates, local times, time zones, and reservation timestamps are represented and exchanged.

Rule 6 — Secure access

Restaurant staff must not be able to manage another restaurant's information without authorization. Customers must not be able to retrieve or alter another customer's private reservation data.

Rule 7 — Reliable error handling

If the network is unavailable, the backend rejects a request, or a reservation conflicts with another booking, both applications should handle the error without showing misleading success messages.

Rule 8 — Clear ownership of functionality

We should distinguish which functionality belongs to the mobile app, which belongs to the desktop software, and which must be implemented in the backend.

This will prevent duplicated work and make testing easier.

9. Suggested Development and Integration Plan

I suggest that we approach the project in stages.

Stage 1 — Agree on the shared requirements

Before implementing the integration, we should agree on the minimum feature set for the first release, the customer journey, the restaurant workflow, and the responsibilities of each application.

Stage 2 — Agree on the data structures

We should define the restaurant, table, customer, reservation, and availability models.

We should also decide which fields are mandatory, how statuses work, and which information is private.

Stage 3 — Agree on the backend and API

We need to decide whether a shared backend already exists or must be developed, which technology will power it, where the database will reside, and how both applications will authenticate and exchange data.

If the backend is being developed by another team member, we should coordinate these requirements with that person as well.

Stage 4 — Develop the applications independently

I can continue implementing the Android screens, navigation, and customer workflows. You can continue implementing the restaurant desktop interface and management functionality.

We should use agreed sample data or mock responses where necessary so that both applications can progress before the live integration is ready.

Stage 5 — Integrate the applications

Once the API is available, we can connect both applications to the same backend and replace mock data with real responses.

Stage 6 — Test complete workflows

We should test scenarios that cross the boundaries between the applications.

For example:

1. A restaurant is added or updated in the desktop software.
2. The restaurant appears in the Android application.
3. Restaurant staff update table availability.
4. The customer app reflects the update.
5. A customer submits a reservation.
6. The reservation appears in the desktop software.
7. Restaurant staff update the reservation.
8. The customer app reflects the new status.

We should also test simultaneous booking attempts, invalid input, network failures, and unauthorized access.

Stage 7 — Prepare the release

After the integrated system passes the agreed tests, we can address remaining defects, improve performance, prepare the demonstration, and document the final workflow.

10. Questions We Need to Resolve Together

To make sure I develop the mobile app in a way that matches your desktop software, I need us to discuss the following points.

1. **Backend:** Are you developing the backend, is another team member responsible for it, or does a working backend already exist?
2. **Technology:** What programming language, framework, and database are you using for the desktop software? What backend technology and database will the complete system use?
3. **Connectivity:** How will the desktop software communicate with the backend? Will the customer app and desktop software use the same API?
4. **Restaurant identity:** How will each restaurant be identified, and how will restaurant staff authenticate and access their own restaurant's data?
5. **Table management:** Will the desktop software manage individual tables, total seating capacity, or both?
6. **Availability:** Will table availability be updated manually by restaurant staff, calculated from table and reservation records, or handled through another workflow?
7. **Waiting times:** Will staff enter estimated waiting times, or will the system calculate them?
8. **Reservations:** Will reservations be confirmed automatically when a suitable table is available, or must restaurant staff approve them?
9. **Reservation lifecycle:** Which statuses will be supported, and which actions can customers and restaurant staff perform?
10. **Live updates:** Will the Android app refresh information through ordinary API requests, periodic polling, WebSockets, or another mechanism?
11. **Customer accounts:** What authentication method will we use, and which customer details are required?
12. **Version 1 scope:** Which features are essential for the initial demonstration and release, and which can be postponed to a later version?
13. **Development environment:** How will we share API documentation, sample responses, test credentials, and integration builds without exposing production secrets?

We do not necessarily need to implement every advanced feature in the first version. However, we should agree on the basic architecture and data contracts before either application becomes dependent on assumptions made by the other developer.

11. Final Objective

My objective is to deliver a functional Android customer application that makes restaurant discovery, availability checking, and reservations straightforward.

Your objective is to deliver the restaurant desktop software that allows restaurant staff to manage the information and operations needed to support that experience.

The shared backend connects our work.

The customer application should not need to understand the internal implementation of the desktop software. Similarly, the desktop software should not need to know how the Android interface is designed. Both applications should communicate through agreed data models and API contracts.

This separation will allow us to develop independently while building one integrated platform.

It will also give us a foundation for future improvements, such as better availability updates, additional restaurant-management features, notifications, analytics, and other customer conveniences.

The goal is not merely to create two applications that look functional when opened separately. The goal is to create one system in which a change made by restaurant staff can be reflected in the customer experience, and a reservation made by a customer can become an actionable record for the restaurant.

That is the relationship we need to design and test together.

Prepared by Anas Afzal

ASANNI — Customer Experience, Android Development, and Application Integration

Document purpose: To explain the customer-side development work, establish the relationship between the mobile and desktop applications, and identify the technical decisions required for successful integration.